

A Log Buffer-Based Flash Translation Layer Using Fully-Associative Sector Translation

발표자 서혜원

발표날짜 2026.08.06

INDEX

3.2 The Architecture

01

3.3 & 3.4 Handling Write&Merge Operations

02

3.5 An Analytical Comparison to BAST

03

3.6 O-FAST

04

4. Performance Evaluation

05

5. Conclusion

06

Think

07

FAST의 2종류 log blocks

SW log block

: Sequential writes 받는 log blocks

RW log block

: Random writes 받는 log blocks

Sequential 전용 log block을 두는 이유:

- flash memory system에서 추적된 workload 분석 시, sequential write多
- switch 연산을 최대한 수행하기 위해

Flash memory system에서 sequential writes가 많은 이유

1. 파일 특성
2. os의 효율적인 관리
3. buffering 기술

FAST의 2종류 log blocks

SW log block

: Sequential writes 받는 log blocks

RW log block

: Random writes 받는 log blocks

FAST의 log blocks: sector-level mapping table을 두어 관리

- **SW log block**
 - 어떤 로그 블록이 어떤 논리 블록에 대응되는지
 - 현재 로그 블록에 몇 개의 섹터가 저장되어 있는지
- **RW log block**
 - 어떤 논리 섹터가 어느 로그 블록의 어떤 offset에 위치하는지
 - 단순히 순차적으로 섹터를 채우는 방식 사용

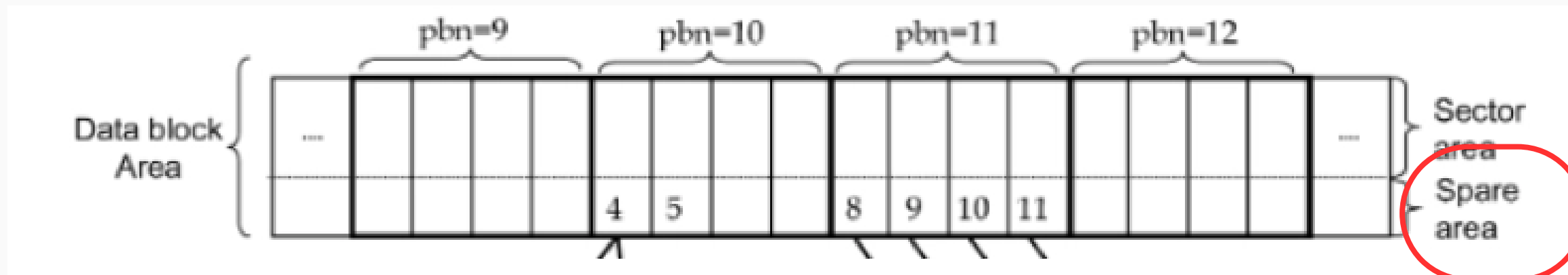
Metadata&Data Consistency Issue

Block-level mapping 방식

- BAST와 같은 방식으로 일관성 유지;
 - 1. map block이라 불리는 전용 블록에 저장 <이중으로 저장되어 영구저장>
 - 2. SRAM에 캐싱 <휘발>

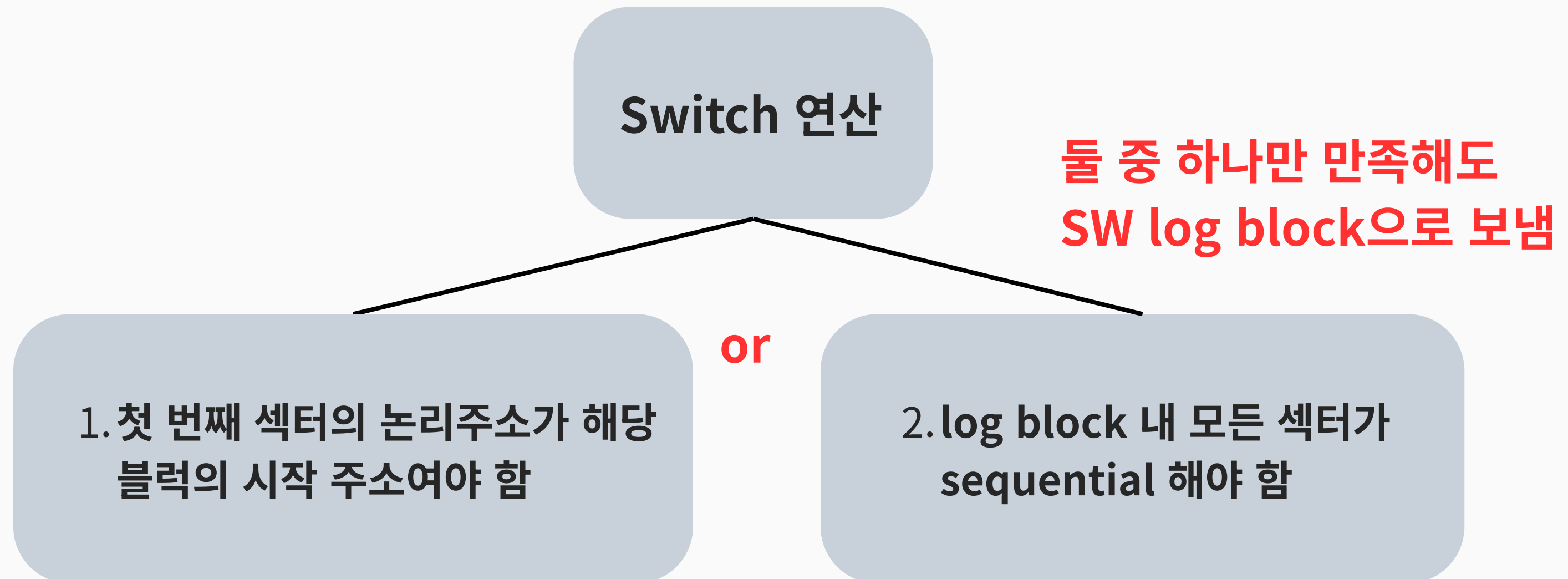
Sector-level mapping 방식

- 물리 섹터의 space area에 기록하는 방식으로 일관성 유지;
 - SRAM에서 정보가 날아갔을 경우 space area에 기록해놓은 정보 이용



Handling Write&Merge Operations

<Overwrite일 시 해당 섹터를 SW/RW 중 어디로 보낼 지 결정>
→ log block에 삽입하는 것이 switch를 유발하는지 검사



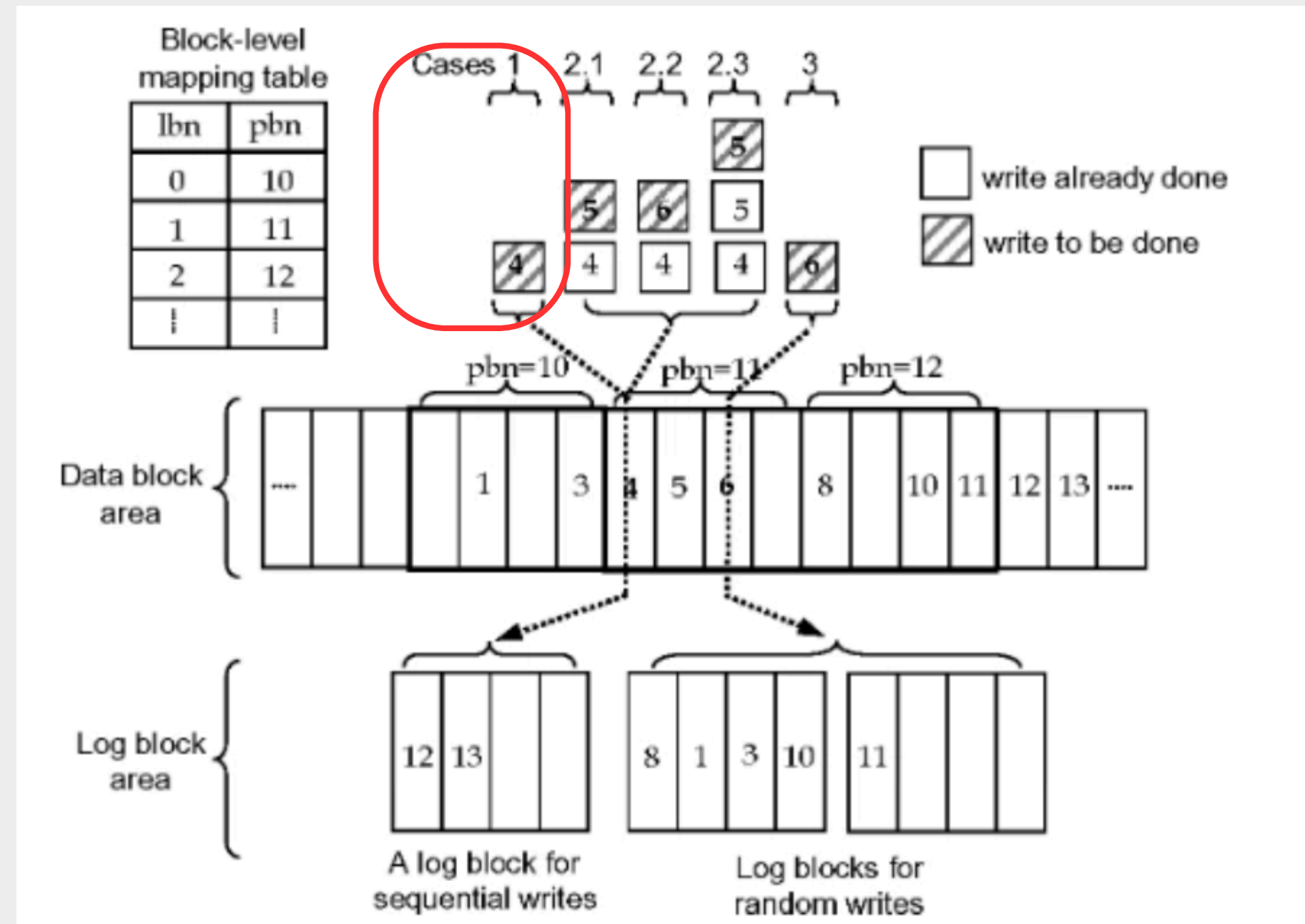
Handling Write&Merge Operations

Data block에서 충돌 발생 3가지 경우 - case1:

- request: sector4 (offset=0)
 - switch 필요조건 1에 해당
 - => SW log block으로 이동

<sw log block에 이미 data가 있을 경우의 merge>.

→ 기존의 merge 방식 가능 But,
이 경우 sw log block에 12 13 순차로 차 있음
=> **SW merge 최적화 대상!**



SW merge 최적화

<기존 merge 방법>

1. 새 블록에다 로그 블록의 12 13 복사
2. 원본 데이터 블록의 14 15 복사
3. block level mapping table에 lbn3의 pbn을 새 블록 번호로 교체
4. 원본 data block erase
5. SW log block erase

=> erase 2번, 복사 4번

<SW merge 최적화 방법>

1. 섹터 14 15만 원래 data block에서 sw log block으로 복사
2. block-level mapping table에 sw log block을 lbn3의 새 data block으로 교체
3. 원본 data block만 erase

=> erase 1번, 복사 2번

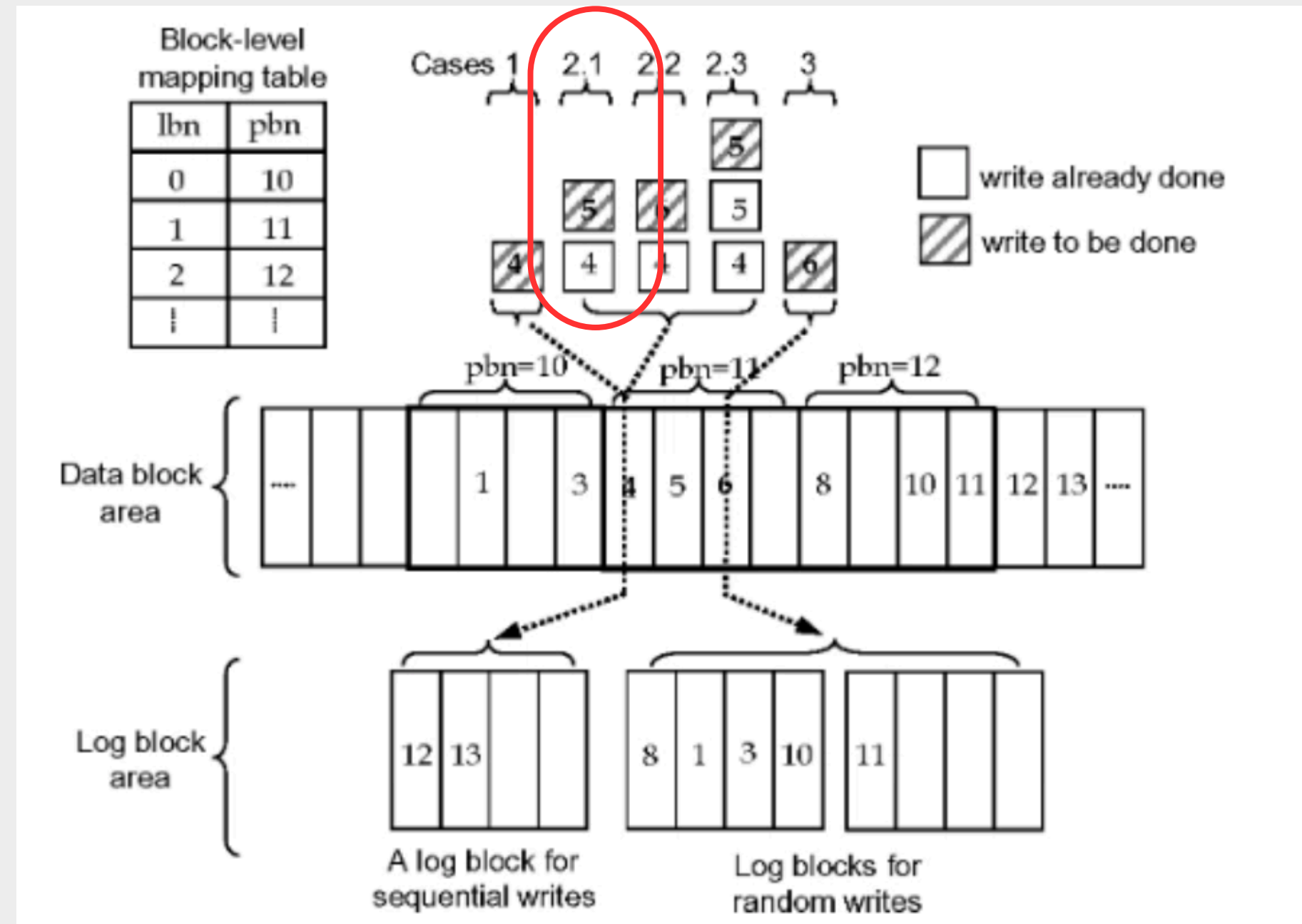
Handling Write&Merge Operations

Data block에서 충돌 발생 3가지 경우 - case2.1:

- request: case1 이후, sector5
 - switch 필요조건 2에 해당
 - => SW log block으로 이동

<sw merge 최적화 과정을 거침

→ sector4 바로 뒤에 sector5 append>

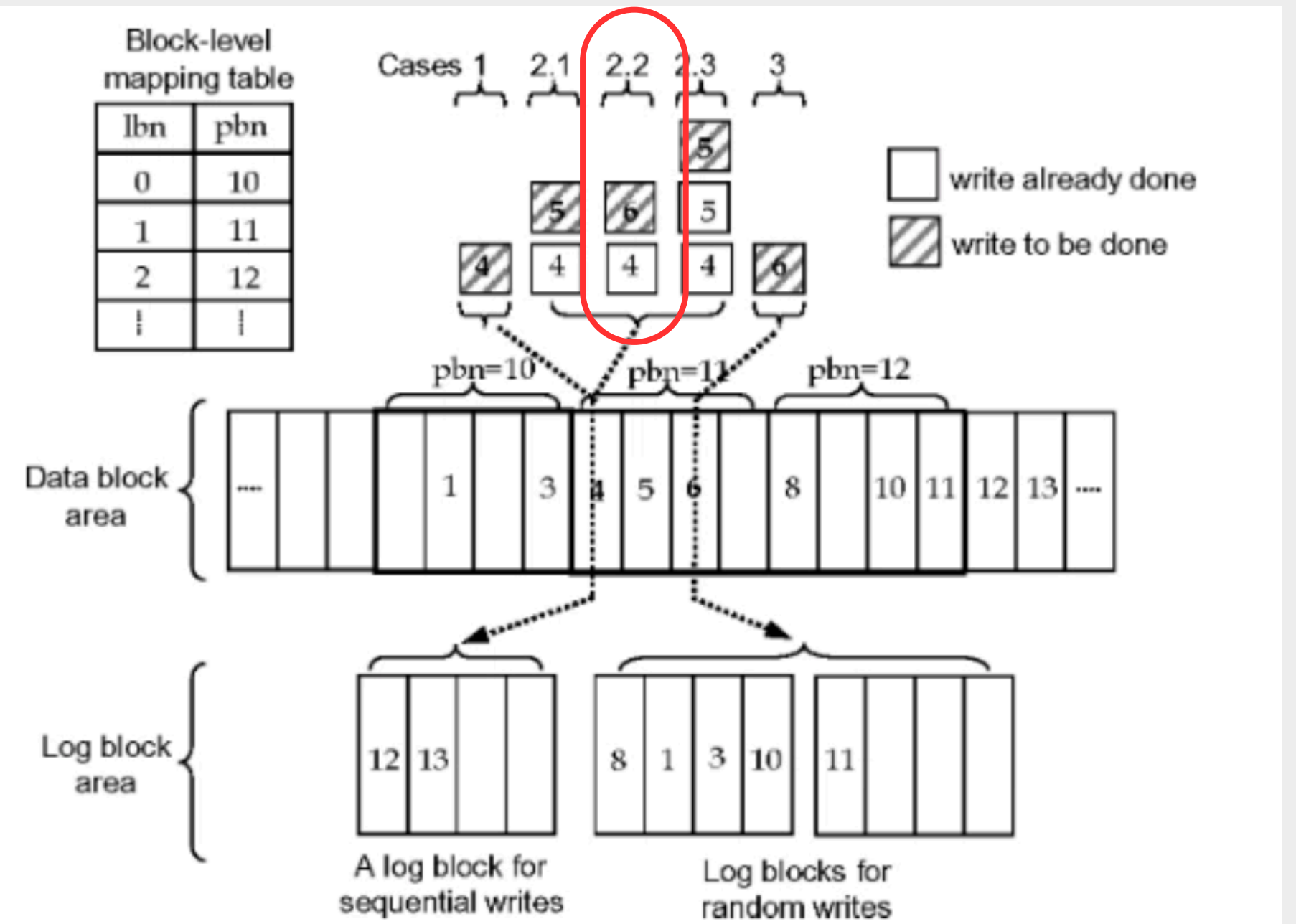


Handling Write&Merge Operations

Data block에서 충돌 발생 3가지 경우 - case2.2:

- request: case1 이후, sector6
 - switch 필요조건 1, 2 모두 실패

<sector4 이후 sector6이 왔으므로 순차가 끊김
→ **sector6은 RW log block에 append**>



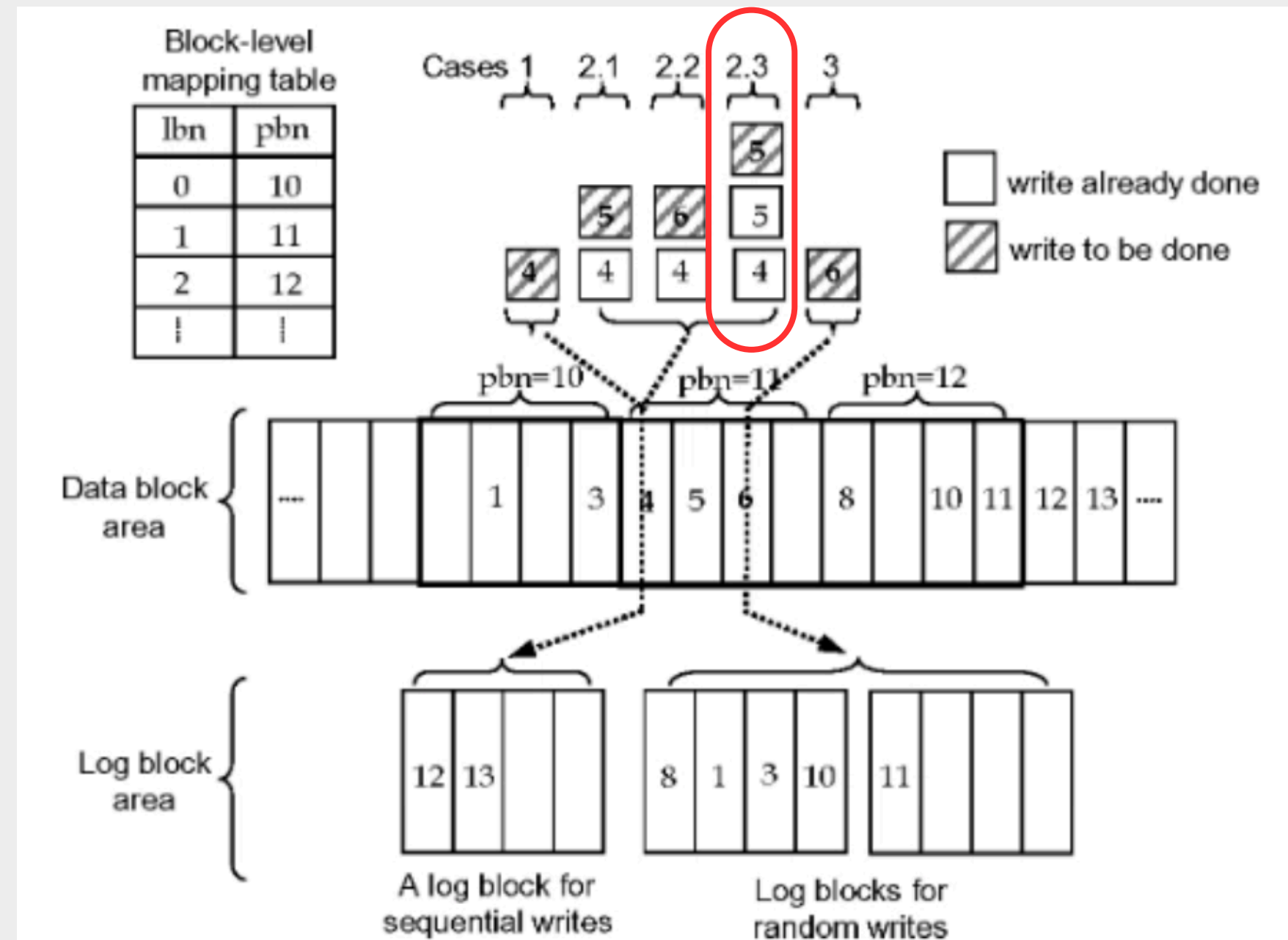
Handling Write&Merge Operations

Data block에서 충돌 발생 3가지 경우 - case2.3:

- request: case1 이후, sector5, sector5
 - switch 필요조건 2만족, 필요조건 2 실패
 - 첫번째 sector5 요청: switch 필요조건 2만족
 - SW log block으로 이동
 - 두번째 sector5 요청: 필요조건 2 실패
 - RW log block으로 이동

첫번째 sector5 요청: <sw merge 최적화 과정을 거침 → **sector4 바로 뒤에 sector5 append**>

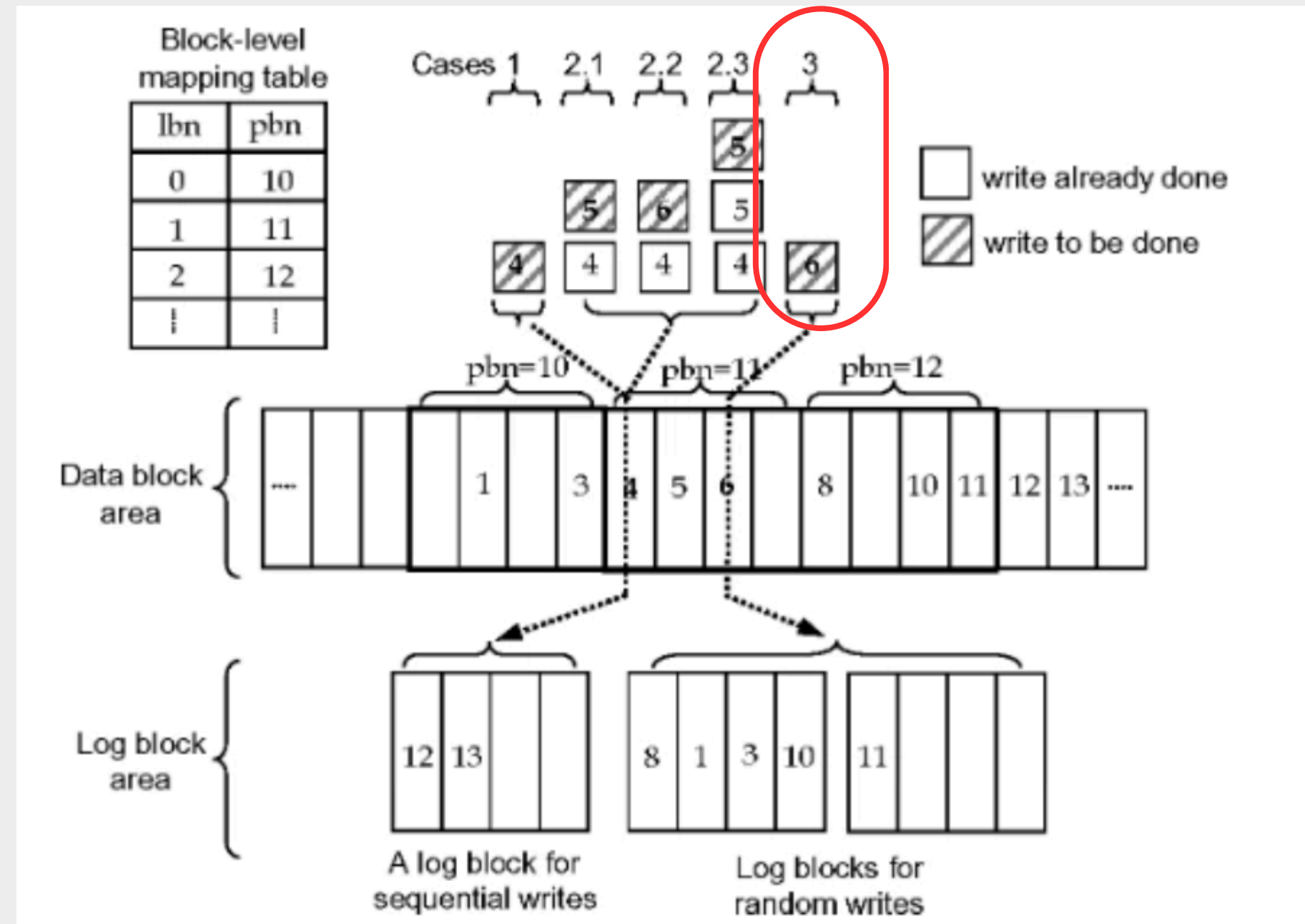
두번째 sector5 요청: <순차가 끊김
→ **sector5는 RW log block에 append**>



Handling Write&Merge Operations

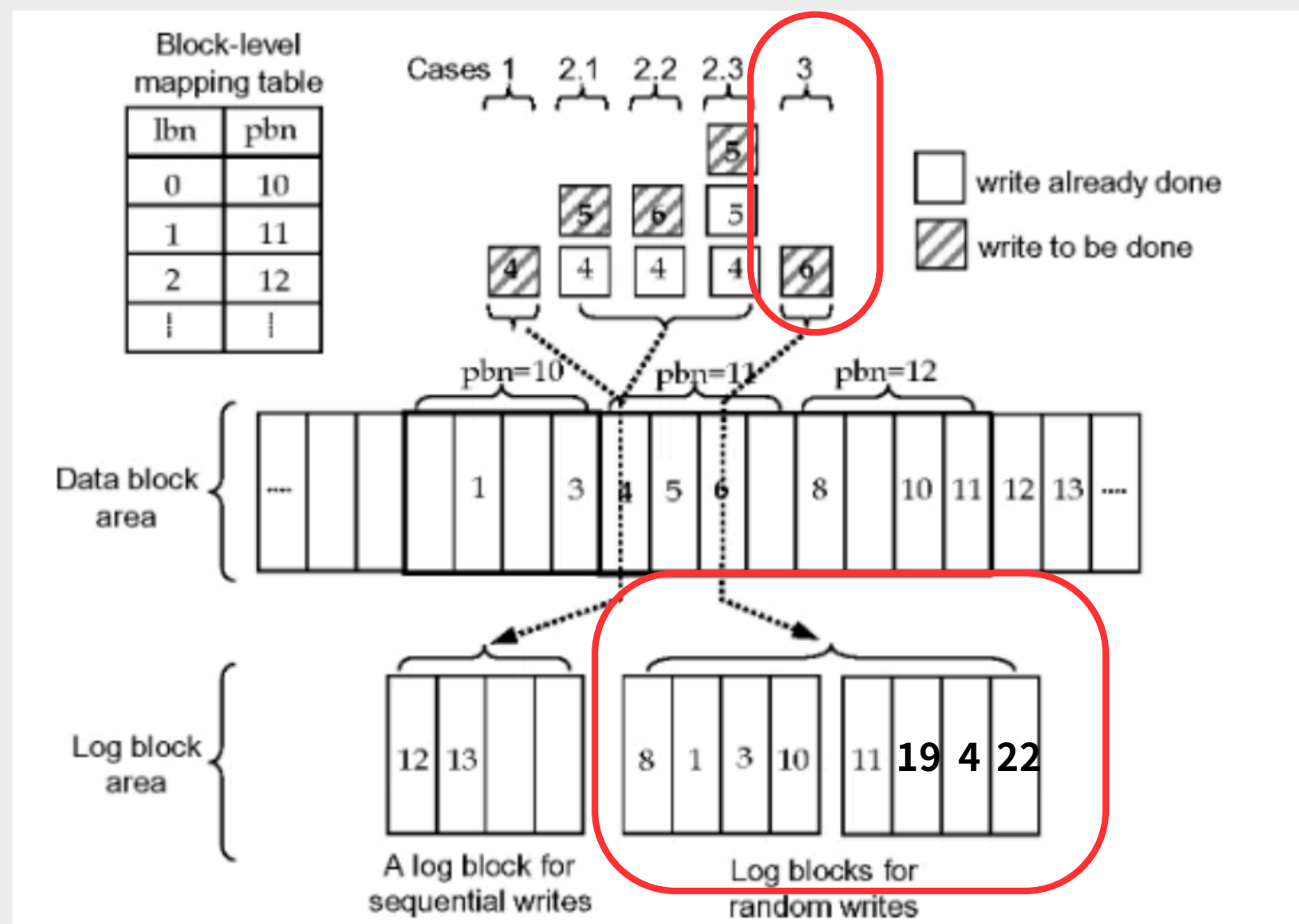
Data block에서 충돌 발생 3가지 경우 - case3:

- request: case1 이후, sector6
 - switch 필요조건 1, 2 모두 실패
 - => RW log block으로 이동

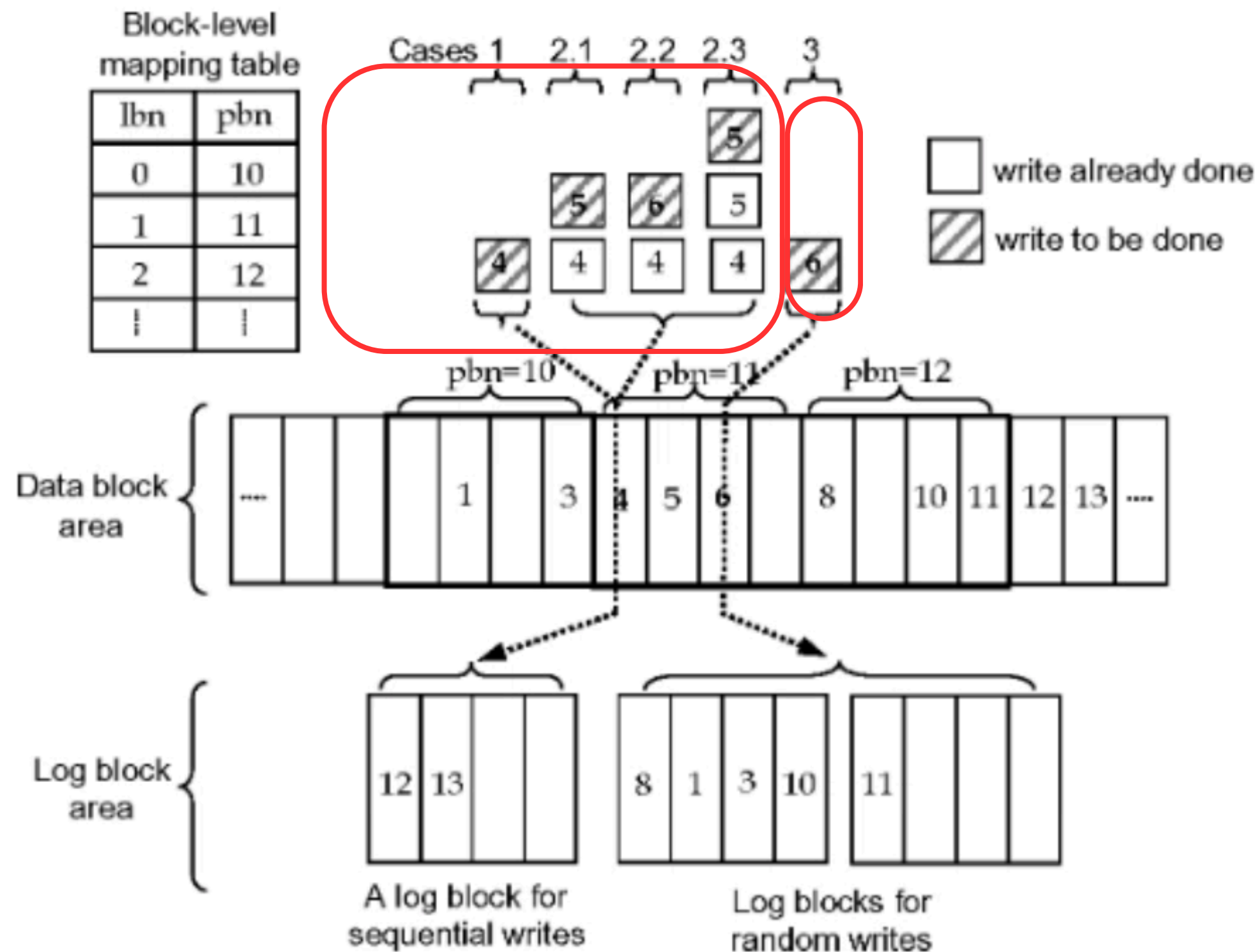


Handling Write&Merge Operations

If) RW log block에 append할 때 RW log block이 다 차 있다면?



Handling Write&Merge Operations



BAST vs FAST

- 핫 블록 수= n_{hb} / 로그 블록 수= n_{lb} / 블록 당 섹터 수= n_{spb}

<대용량 순차쓰기에 대해선 성능이 비슷하므로, Random write 패턴에 집중>

Case	조건	BAST	FAST	비고
1	$n_{hb} = n_{lb}$	약 $2 \times n_{lb}$ 번 erase	약 $2 \times n_{lb}$ 번 erase	두 방식 성능 비슷
2	$n_{hb} = 1$	n_{spb} 번째 쓰기마다 merge 발생	$n_{lb} - 1$ 번 erase 절 약	로그 블록이 다 찰 때까지 write 수용 가능해서 FAST가 유리
3	$n_{hb} > n_{lb}$ (현실적 상황)	n_{lb} 가 n_{hb} 와 같아지기 전까진 개선 없음	로그블록 개수 늘릴 수록 이득 커짐	확장성 차이가 가장 두드러지는 케이스
4	$1 < n_{hb} < n_{lb}$	n_{hb} 가 n_{lb} 에 근접할수록 FAST에 근접, n_{lb} 넘는 순간 급격히 나빠짐(thrashing)	거의 일정	BAST가 FAST를 따라잡는 구 간

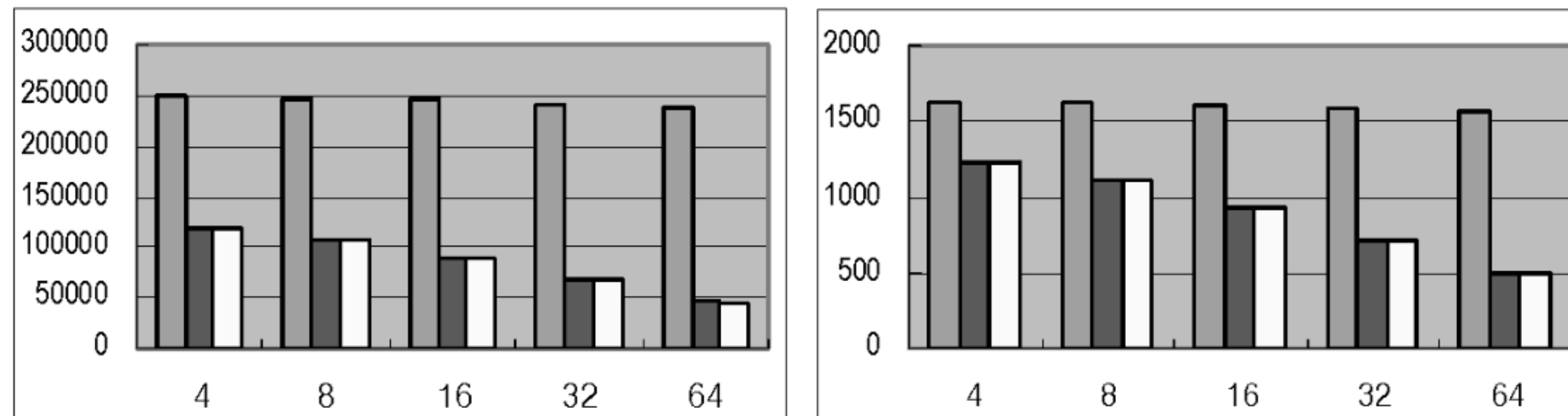
Performance Evaluation

디지털카메라 2개 회사, 리눅스, symbian, 랜덤 총 5개의 trace로 시뮬레이션

1. FAST가 모든 패턴에서 BAST를 이김
2. log block 개수가 적을 수록 FAST 성능이 증명됨
3. 워크로드가 랜덤할수록 FAST 우위가 커짐

■ BAST ■ FAST □ O-FAST

X-axis : # of log blocks, Y-axis in left side : erase count, Y-axis in right side : elapsed time(secs).



(e) Pattern E: Random 제일 극적인 경우

Conclusion

FAST는 fully-associative sector translation으로 BAST의 block-level associativity 한계를 극복한 log buffer 기반 FTL이다.

- log block thrashing 회피
- merge연산 최대한 지연
- rw log block merge에서 불필요한 merge를 skip

=> FAST는 log block 4~8개만으로 BAST가 30개 이상 필요로 하는 성능을 냄

Think

- FAST 방식을 사용할 시, fig5에서 이미 sw log block에 12 13이 차있는데 request 순서가 4 14 15 이런식으로 온다면 merge횟수가 너무 많아지지 않을까?
 - 실제 워크로드에선 저런 input이 드물다. 보통 sequential한 워크로드를 가짐
- 만약 lsn의 lbn을 계산했을때 같은 블록안에 있으면 merge할때 같이 할 수는 없나? (case2.2 lsn6에 대해서)
 - 알고리즘 구조상 merge 실행과 신규 요청 배치가 분리된 단계여서 논문의 FAST 알고리즘 그대로는 해결하지 못 함

THANK YOU